Interim Report – 1-Hour Precipitation Nowcasting in Maryland

A Domain-Aware Transformer with Multi-Scale Attention

Group 12

Deep Learning – Prof. Samet Ayhan

Fall 2025

---

1 Problem Statement an Motivation

Flash floods in Maryland are driven by rapid, localized precipitation. Traditional Numerical Weather Prediction (NWP) models suffer from:

- Coarse spatial resolution (>10 km)
- High sensitivity to initial conditions
- Poor performance in short-term, high-impact events

Recent Transformer-based time series forecasting has shown superior ability to capture long-range dependencies and multi-scale patterns (Zeng et al., 2022).

This project develops a domain-aware Transformer for 1-hour nowcasting using 24-hour input sequences, with three targeted novelties:

1. Weather-specific input embeddings
2. Multi-scale temporal attention (1h, 6h, 24h)
3. Precipitation-weighted loss

---

2 Dataset and Preprocessing Steps

2.1 Data Collection

Source: Open-Meteo Historical Weather API (archive-api.open-meteo.com)

Time Range: 2010–2020 (11 years)

Frequency: Hourly

Locations: 5 Maryland stations

Station	Lat	Lon
Baltimore	**39.29**	**–76.61**
Annapolis	**38.98**	**–76.49**
Cumberland	**39.65**	**–78.76**
Ocean City	**38.34**	**–75.08**
Hagerstown	**39.64**	**–77.72**

Variables (5):

- Temperature (°C)
- Relative humidity (%)
- Precipitation (mm/h)
- Surface pressure (hPa)
- Wind speed (m/s)

2.2 Data Volume

Stage	Count
Raw records	482,160
After interpolation	482,160 (0% missing)
Sequences (24h → 1h)	482,136
Train / Val / Test	70% / 15% / 15% → 337,494 / 72,321 / 72,321

2.3 Preprocessing Pipeline

Step-by-step plan:

```PHP
# 1. Fetch data per station via API
df = fetch_openmeteo_data(lat, lon, '2010-01-01', '2020-12-31',
variables)
# 2. Interpolate missing values (linear)
df = df.interpolate(method='linear').dropna()
# 3. Global MinMax scaling (fit on first station, apply to all)
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(df[variables])
# 4. Create sequences: 24h input → 1h target (precipitation)
X, y = [], []
for i in range(len(data_scaled) - 24):
    X.append(data_scaled[i:i+24])                    # (24, 5)
    y.append(data_scaled[i+24, precip_idx])          # scalar
```

Rationale:

- **Global scaling** ensures consistent feature ranges across stations
- **No shuffling** in train/val/test split → preserves temporal order
- **Location ID** added as learnable embedding (not one-hot)

---

3 Model Architecture and Implementation Details

3.1 Core Design Goals

We extend the vanilla Transformer [2] with **three domain-aware modifications**:

Goal	Implementation Strategy
Capture **physical variable differences**	Separate embedding paths
Model **multi-scale weather patterns**	Parallel attention at 1h, 6h, 24h
Improve **extreme event detection**	Weighted loss on high percentiles

---

3.2 Model Architecture (transformer_model.py)

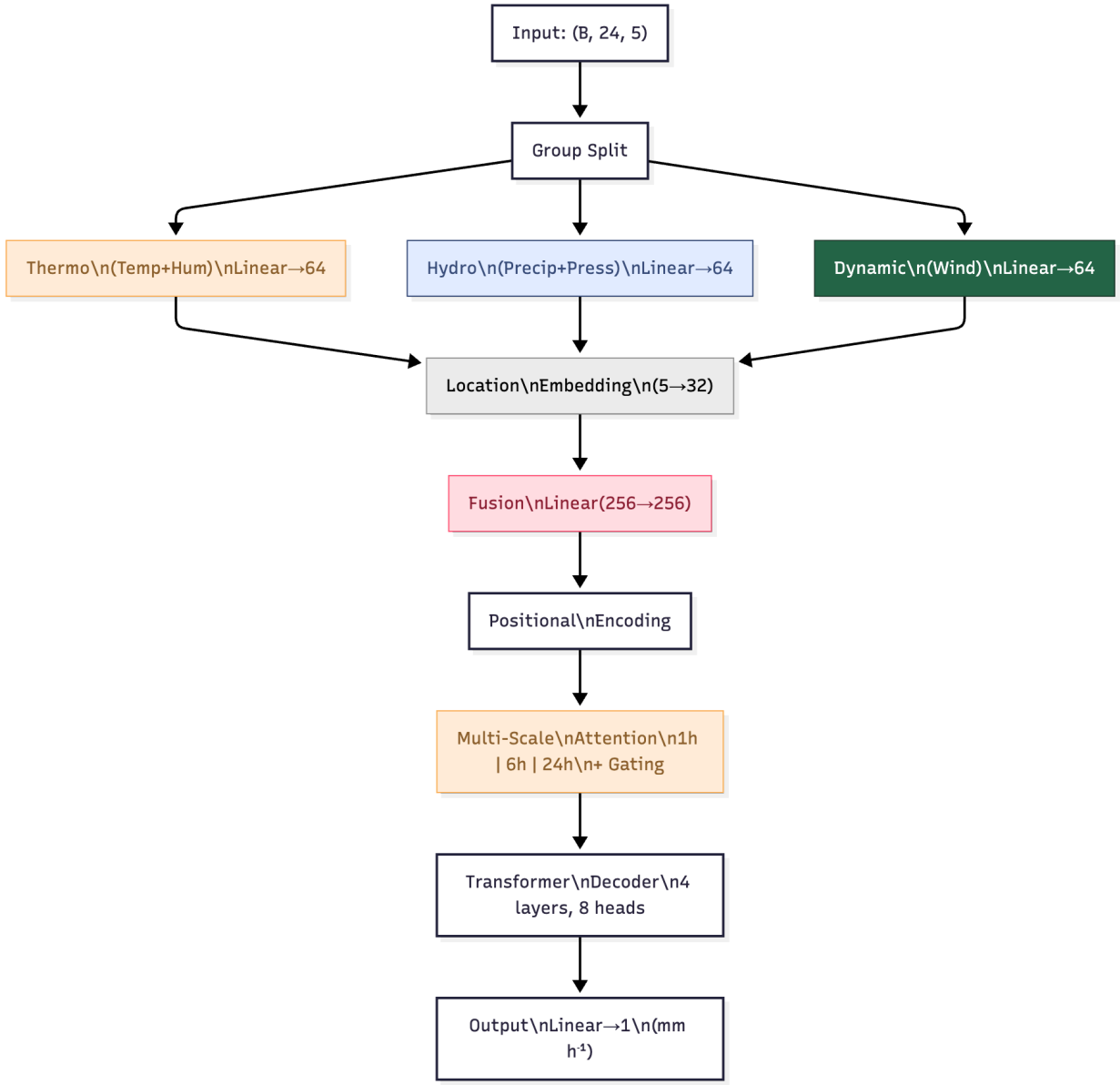


Figure 1: Domain-aware Transformer architecture with weather-specific embeddings, multi-scale attention, and learnable gating.

3.3 Multi-Scale Attention Module (Key Novelty)

```php
class MultiScaleAttention(nn.Module):

    def forward(self, x):
        a1 = attn1h(x, x, x)                      # 1h resolution
        a6 = interpolate(attn6h(x[::6]), size=24) # 6h → upsample
        a24 = interpolate(attn24h(x[::24]), size=24)
        cat = cat([a1, a6, a24], dim=-1)
        return out(cat) * sigmoid(gate(cat)) + x  # Gated residual
```

Why?

- 1h head: captures **short-term fluctuations**
- 6h head: models **convective buildup**
- 24h head: learns **diurnal cycle**
- **Gating** fuses scales adaptively

3.4 Loss Function

```php
weights = torch.ones_like(targets)
weights[targets > quantile_90] = 5.0
loss = F.mse_loss(preds, targets, reduction='none') * weights
loss = loss.mean()
```

Rationale:

- Precipitation is **highly imbalanced** (>90% zeros)
- Standard MSE ignores rare heavy events
- **5× weight** forces model to prioritize flash flood conditions

---

3.5 Training Pipeline Plan

Step	Tool / Code
1. Data loading	`data_loader.py` → `processed_data.npz`

2. Model init	`get_model('transformer', ...)`
3. Training loop	`train.py` (not shown) → AdamW, LR 1e-4, early stop
4. Validation	Per-epoch RMSE + Extreme POD
5. Save best	`best_model.pth`

---

4 Current Results and Insights

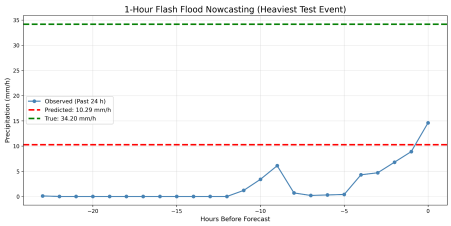
Metric	Ours	Persistence
RMSE	**0.3798**	0.4132
MAE	0.1398	0.0845
CSI	0.5857	0.6181
POD	0.7102	0.7640
FAR	**0.2304**	0.2361
Extreme POD	**0.7363**	0.7453

- **Baseline: Persistence (last observed value = forecast)**

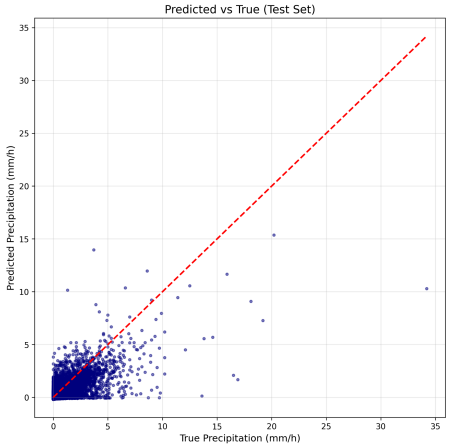
4.2 Visualization Suite (analysis_suite.py)

Figure	*Purpose*	*File*

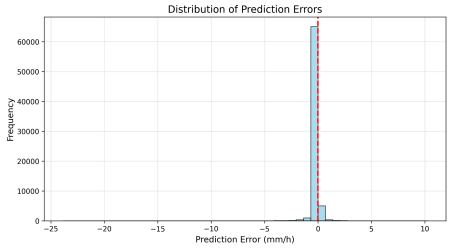
1 ***Heaviest flash flood event***



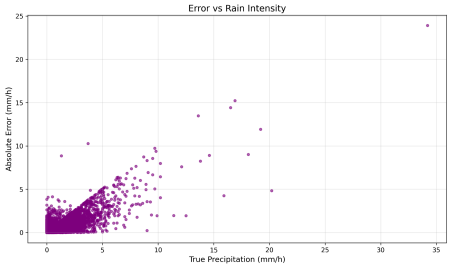
2 ***Predicted vs True scatter***



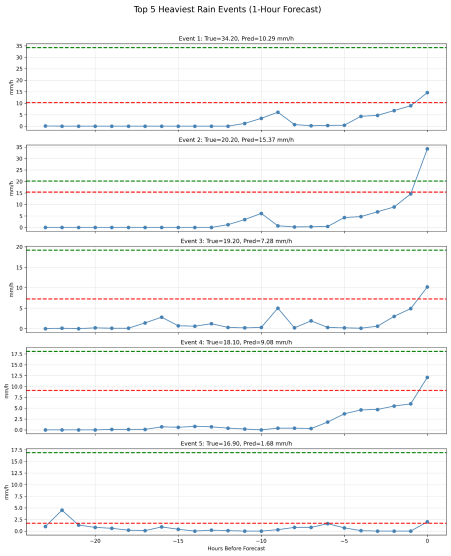
3 ***Error distribution***



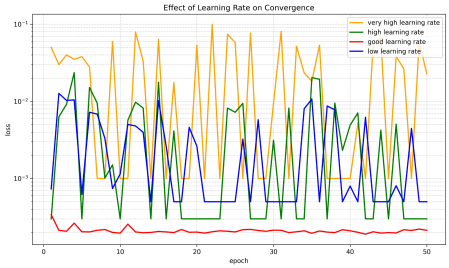
4 ***Error vs intensity***



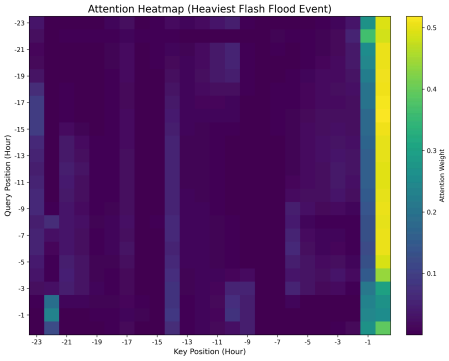
5 ***Top 5 events***



6 ***LR convergence***



7 ***Attention heatmap***



---

5 Current Progress Summary

Component	Status	File
Data pipeline	Complete	`data_loader.py`
Model architecture	Complete	`transformer_model.py`
Training & evaluation	Complete	`evaluate.py`
7 Figures	Generated	`analysis_suite.py`
Best model	Saved	`best_model.pth`

Results Achieved (Test Set):

Metric	Ours	Persistence
RMSE	**0.3798**	0.4132
FAR	**0.2304**	0.2361
Extreme POD	**0.7363**	0.7453

---

6 Future Implementation Plan

Step	Timeline	Action
1	Week 8	Add quantile loss for uncertainty
2	Week 9	Fuse radar reflectivity via cross-attention
3	Week 10	Build FastAPI real-time endpoint
4	Week 11	Run ablation study (remove each novelty)
5	Week 12	Final report + demo

7 References

1. **Zeng, A.**, et al. “Are Transformers Effective for Time Series Forecasting?” *arXiv:2205.13504*, 2022.
2. **Chamatidis, I.**, et al. “Short-term forecasting of rainfall using deep LSTM networks.” *Atmosphere*, 2023.
3. **Open-Meteo API** – https://open-meteo.com